

TULIP: Finding Backdoors with Protocol-Aware Fuzzing

Alexandre Doyen

Université Paris-Saclay, CEA, List
Paris-Saclay, France
alexandre.doyen@cea.fr

Michaël Marcozzi, Sébastien Bardin

Université Paris-Saclay, CEA, List
Paris-Saclay, France
first.last@cea.fr

Jean-Yves Marion

LORIA, Université de Lorraine, CNRS, Inria
Nancy, France
Jean-Yves.Marion@loria.fr

Abstract—Code-level backdoors are hidden functionalities embedded in source code that enable unauthorized access or privilege escalation when triggered by specific, secret inputs. A typical example is the injection of hard-coded credentials into a file server’s codebase, allowing attackers to compromise all deployed instances globally. Such backdoors have been observed in real-world supply-chain attacks targeting open-source software and embedded network daemons. Greybox fuzzing—an automated testing technique that combines randomized input generation with lightweight instrumentation—has proven effective for discovering memory-safety vulnerabilities. However, its application to backdoor detection remained largely unexplored until the recent emergence of the ROSA tool, which demonstrated that general-purpose fuzzers like AFL++ can be repurposed for this task. While ROSA significantly advanced automation compared to earlier approaches, the current state of the art still suffers from a critical limitation: the generation of false alarms that require manual vetting. To address this, we present TULIP, a protocol-aware greybox fuzzing framework specifically designed to detect backdoors in implementations of application-layer network protocols, such as web and file server programs. By specializing in such programs—where most known backdoors reside—TULIP can leverage dedicated protocol models to identify backdoor-triggering behaviors without producing false alarms. These models can also guide the fuzzer towards input sequences more likely to expose backdoor logic. We evaluate TULIP across 14 backdoors affecting 4 widely used protocols (HTTP, FTP, POP, and IMAP), including 7 authentic samples from real-world attacks, four of which are CVEs. Our results show that TULIP consistently avoids false alarms and, in most cases, detects backdoors faster than ROSA.

Index Terms—Backdoor Detection, Model-based Fuzzing, Protocol Implementations

This work was supported by the French National Research Agency (ANR), under JCJC programme (with reference ANR-22-CE39-0012- 01) and under Plan France 2030 (with references ANR-22-PTCC-0001, ANR-22-PECY-0007, ANR-22-PECY-0009).

I. INTRODUCTION

Context. A *code-level backdoor* [17], [10] refers to a covert mechanism deliberately embedded within a program’s source code. This hidden functionality allows individuals who are aware of its existence to input a specific trigger value, thereby activating unauthorized privilege escalation or gaining illicit access to forbidden resources. A typical example involves embedding hard-coded credentials into the codebase of an FTP server program, which can be exploited to compromise

all instances of the program globally. Real-world incidents of software supply-chain attacks have resulted in such backdoors being introduced into widely used open-source software, including the HTTP module of PHP [6], ProFTPD [14] and vsFTPd [4]. Similar vulnerabilities have also been identified in network daemons integrated into the firmware of mainstream router devices [19], [12], [3], [2].

Greybox fuzzing [7] has become a cornerstone of automated software testing. It employs search heuristics [8] to systematically generate test inputs and utilizes lightweight test oracles [1]—typically crash detectors—to identify anomalous runtime behavior during testing. Contemporary greybox fuzzing frameworks—such as the community-driven AFL++ [5]—have garnered significant attention in both academic and industrial circles due to their demonstrated effectiveness [5] in uncovering memory-safety vulnerabilities. Yet, in the context of code-level backdoors, the issue is different: backdoors do not manifest as crashes but as hidden yet valid functionalities, triggered by secret input values. Detecting such covert mechanisms requires fuzzers not only to explore input spaces efficiently but also to be combined with oracles capable of distinguishing legitimate from backdoored behaviors.

Problem. A recent approach, known as ROSA [10], has demonstrated that a general-purpose fuzzer like AFL++, augmented with a heuristic test oracle, can be repurposed to reveal backdoors across diverse software systems. By relying on greybox fuzzing, ROSA marked a substantial advancement in automation compared to the earlier backdoor detection approaches [14], [15], [16], [18], which all still required some manual reverse-engineering of the analyzed code. While being able to analyze many diverse kinds of programs was another notable contribution of ROSA, it may also possibly represent a missed opportunity for beneficial specialization. Indeed, a significant portion of documented code-level backdoors [14], [15], [16], [18], [17], [10] are found in implementations of standard application-layer network protocols, such as HTTP or FTP server programs. The code of these programs, which are commonly deployed at the interface between the public Internet and private infrastructures, is indeed a prime target for backdoor injection attacks. While false alarms — to be manually discarded by a human expert — are an important limitation of ROSA and other previous backdoor detection

approaches, specializing on protocol implementations could offer the opportunity to construct protocol models, usable as specification-based oracles, to identify backdoored behaviors without false alarms. Such models could also guide general-purpose fuzzers so that they trigger backdoors faster. However, to date, no such protocol-aware fuzzing approach [20] for backdoors has ever been designed and its resulting benefits are actually unknown.

Goal and challenges. We aim at developing a backdoor-oriented protocol-aware fuzzer, capable of detecting backdoors in protocol implementations *faster* than with a raw general-purpose fuzzer and *without false alarms*. Achieving this objective, however, entails addressing three key challenges. First, application-level network protocols have huge and complex textual specifications. It is thus essential to provide a means of abstracting these specifications into compact and actionable models. These models should describe only the protocol features diverted by backdoors and do it in a way that enables the automatic recognition of the backdoored behaviors. Second, using the model to make backdoor detection faster requires to build a new fuzzing guidance mechanism. This mechanism should combine symbiotically (1) the state-of-the-art guidance mechanisms already present in greybox fuzzers with (2) the preexisting knowledge of how backdoors should behave, brought by listing the ways a protocol implementation could diverge from the model. Third, in order to ease the evaluation of widely deployed protocol implementations without requiring manual modeling, it is necessary to offer validated pre-built models for standard protocols.

Proposal. We present TULIP, a protocol-aware greybox fuzzing approach for detecting code-level backdoors in network protocol implementations. At its core lies the Command Specification Language (CSL), a modeling language that abstracts the protocol specification for backdoor detection. A CSL model has two components: (i) a lightweight state machine that encodes the authentication workflow, and (ii) a specification of the expected external behavior of the protocol’s commands, expressed as admissible system call patterns. To ensure broad applicability, we provide complete CSL models for widely used protocols such as HTTP, FTP, POP, and IMAP, and validate them across multiple implementations. Given such a model, TULIP automatically synthesizes adversarial command sequences designed either to violate the authentication flow or to deviate from expected command behaviors. These sequences are executed against the target protocol implementation, and any behavioral deviation from the CSL specification is reported as a backdoor. Many backdoors require secret command or argument values to be activated. This makes most adversarial command sequences parametric on one or several unknown activation values. To uncover them, TULIP embeds an augmented version of AFL++ that is explicitly adapted to resolve the missing values in parametric adversarial sequences. This integration allows the fuzzer to systematically explore hidden values while preserving the structure of the surrounding sequence, thereby enabling efficient discovery of backdoor

triggers.

Evaluation. We evaluate TULIP against the state-of-the-art over 14 backdoors, divided into 7 authentic samples from real-world attacks — including 5 CVEs — and 7 synthetic samples. These 14 samples affect 7 programs spanning 4 standard network protocols, and they include the 5 network backdoors previously analyzed in the ROSA study [10]. We perform 20 1-hour runs over each backdoor and measure the backdoor detection speed and the quantity of produced false alarms. Across all the runs, TULIP was able to find the backdoor 220 times (78%) and ROSA 185 times (66%). On average, 10 backdoors were found faster by TULIP, 2 were found faster by ROSA and there was no remarkable speed difference over the 2 others. ROSA produced false alarms for 14 out of 14 backdoors, with an average of 2.9 false alarms per backdoor, while TULIP never produced any false alarms, as expected. To sum up, while TULIP and ROSA do not require any manual reverse-engineering of the analyzed code (making them more automated than earlier approaches), TULIP also removes the need for manually vetting the produced backdoor reports, as its protocol-aware test oracle does not yield false alarms. Regarding speed, TULIP is most of the time faster than ROSA to detect backdoors, but each tool can still be overwhelmingly faster than the other to detect specific backdoors, depending on the backdoor design. Overall, we recommend practitioners giving priority to TULIP¹ for backdoor detection, as it does not produce false alarms to vet manually and is faster on average than ROSA on diverse backdoors. Yet, if enough resources are available, running both TULIP and ROSA could be beneficial, as each can discover some backdoors with significantly less effort than the other.

Contributions. To sum up, our main contributions are:

- 1) TULIP, a **protocol-aware greybox fuzzing technique and tool to detect code-level backdoors** in application-level network protocol implementations. Its components are:
 - A *modeling language* to specify the commands semantics and authentication sequence of network protocols, enabling flagging backdoored behaviors without false alarms.
 - Fully developed *models* for widely adopted protocols —HTTP, FTP, POP, and IMAP— validated across real-world implementations.
 - A unified *mechanism to search for backdoor triggers*, which integrates the generation and execution of adversarial input sequences derived from protocol models with the AFL++ fuzzer. This fuzzer is augmented to efficiently complete the adversarial sequences with the secret commands and arguments required to activate the backdoor.
- 2) An **experimental evaluation of TULIP** over 14 backdoor benchmarks. The tool’s detection speed and false alarm

¹For protocols with no preexisting CSL model, TULIP comes with an additional upfront manual cost for creating the model. We provide off-the-shelf models for common protocols like HTTP, FTP, POP and IMAP.

rate are notably compared **against the state-of-the-art ROSA fuzzing approach**, which relies on vanilla AFL++ and a heuristic oracle, with no specialization for protocol implementations. The results show that TULIP removes the need for manually discarding false alarms and is most of the time faster to detect diverse backdoors.

Impact. By bridging protocol modeling with greybox fuzzing in the context of backdoor discovery, TULIP offers a faster and precise solution for detecting backdoors in network-facing software. This work also contributes a practical backdoor detection tool, which is usable openly for strengthening the resilience of critical network services against covert exploitation.

II. EXPERIMENTAL EVALUATION

A. General overview

We aim to answer the following research question:

- Can TULIP detect the different types of backdoors appearing in implementations of standardized application-level network protocol faster and with less false alarms than state-of-the-art backdoor detection tools?

Experimental setup. We design our experimental setup by carefully adapting the best practices for fuzzing evaluation [9], [13] to the backdoor case:

- We have built a *backdoor detection benchmark* made of 14 backdoors injected in real network protocols implementations. We have included as many backdoors as possible from the ROSARUM [11] benchmark, as it is the only standard benchmark existing in the literature and the one used to evaluate the closest competing approach, namely ROSA [10]. To complete our benchmark, we have first built 7 synthetic backdoors in mail server programs, inspired by our motivating example. Second, we have searched common public vulnerability and exploit databases (like the CVE database) for additional authentic backdoors fitting TULIP’s scope. The 2 backdoors found so were either directly integrated in our benchmark (when the backdoor code was available), or replicated based on the backdoor description (when the code was not available). The final benchmark covers 9 versions of 7 implementations of 4 standard protocols.
- All the experiments were realized on identical Intel® Xeon® 2.2 GHz servers. To *tame the effects of randomness* over fuzzing performance, we perform 20 independent 1-hour runs of each fuzzing-based tool for every backdoor in our benchmark, over 6 CPU cores.

B. Answer to the research question

Compared tools and fuzzing seeds. The only existing fuzzing-based backdoor detection tool is ROSA [10], which combines AFL++ with a dedicated metamorphic oracle, and to which we obviously compare. In our experiments with ROSA, we have copied the experimental setup described in the ROSA paper [10], which includes reusing the seeds employed in that

paper for fuzzing HTTP and FTP implementations. As ROSA had never been used to fuzz POP and IMAP implementations, we have collected dedicated seeds similarly to what the authors of ROSA did for HTTP and FTP. These seeds are available as a part of this paper’s artifact. As stated by the authors of ROSA in their paper, only four older backdoor detection tools do exist, among which only STRINGER [16] is still both available and working, so that we also include it in our comparison. Through static analysis of binary executables, STRINGER deterministically identifies and extracts embedded string literals that are potentially indicative of hard-coded backdoor activation mechanisms.

Experimental protocol. To answer the question, we run TULIP, ROSA and STRINGER to detect the 14 backdoors from our benchmark, then we report the measured detection speed and precision results. Regarding *speed*, as STRINGER is a deterministic tool, we only report whether a part of the backdoor trigger value was correctly extracted from the PUT’s binary and how long it took if applicable. For the two fuzzing-based tools (TULIP and ROSA), we report (1) how many times an input triggering the backdoor was identified by the tool before timing out across the 20 1-hour runs and (2) the average detection time across the runs. To evaluate whether the measured detection time difference between TULIP and ROSA is statistically significant over the 20 runs, we report the obtained *p*-values (Mann-Whitney U statistical test) [9], [13]. A *p*-value of 0.00 means full statistical significance and a *p*-value of 1.00 means a complete absence of any statistical significance. A *p*-value of 0.05 is usually considered as the minimal threshold under which a measured difference can be reasonably considered as statistically significant. Regarding precision, we report the number of *false alarms* produced by each tool. For TULIP and ROSA, this is the number of PUT’s inputs mistakenly reported as triggering a backdoor. For STRINGER, it is the number of strings extracted from the PUT’s binary mistakenly reported as being part of a backdoor trigger before finding the backdoor. As ROSA produces different numbers of false alarms across runs, we report averages.

Results discussion. The obtained results are detailed in Table I. In terms of detection *speed*, STRINGER has all the cons and pros of relying on a lightweight static analysis, as it only detects 4 out of 14 backdoors (against 13/14 for TULIP and 11/14 for ROSA), but it is orders of magnitude faster than the two fuzzing-based tools when detection succeeds.

Across all the 280 1-hour runs, TULIP was able to find the backdoor 220 times (78%) and ROSA 185 times (66%). Among the 14 backdoors, 10 (71%, + notation in the table) were found faster by TULIP, 2 (14%, −) were found faster by ROSA and both tools were on par with each other over the 2 (14%, ~) remaining ones. On average, TULIP is 26% faster than ROSA at detecting the backdoors (19m22s per backdoor for TULIP against 24m18s for ROSA).

By summarizing, TULIP achieves a strong detection capability (recall = 0.78, i.e., it detects the backdoor in 220 out of 280 trials) while maintaining perfect precision (precision

Backdoor	Passed runs		Average detection time (timeout = 1 hour)				False alarms		
	TULIP	ROSA	TULIP	ROSA	(p-value)	STRINGER	TULIP	ROSA [avg]	STRINGER
Authentic backdoors									
ProFTPD 1.3.3	20/20	19/20	6m3s	14m24s	+ (0.01)	5s	0	1.9	345
ProFTPD 1.3.5	20/20	0/20	0s (no fuzzing)	Timeout	+ (0.00)	Not found	0	2.15	469
VsFTPD 2.3.4	20/20	19/20	3m	7m14s	+ (0.00)	Not found	0	3	119
Belkin (HTTP)	11/20	0/20	43m21s	Timeout	+ (0.02)	Not found	0	1.05	0
D-Link (HTTP)	12/20	4/20	38m15s	50m42s	+ (0.04)	Not found	0	3.9	113
PHP (HTTP)	0/20	19/20	Timeout	6m53s	— (0.00)	6m35s	0	1.7	573
Replication of authentic backdoors									
VsFTPD 3.0.5	11/20	0/20	40m48s	Timeout	+ (0.00)	Not found	0	1.2	128
Synthetic backdoors									
Cyrus 3.12.1 (IMAP)	19/20	20/20	18m45s	17m3s	~ (0.92)	5s	0	8.1	864
gmail a. (POP3)	20/20	20/20	0s (no fuzzing)	5m30s	+ (0.00)	Not found	0	1.1	1
gmail b. (POP3)	20/20	18/20	0s (no fuzzing)	9m57s	+ (0.00)	Not found	0	1.6	1
gmail c. (POP3)	20/20	20/20	4m12s	3m39s	— (0.00)	< 1s	0	4.1	1
gmail d. (POP3)	20/20	19/20	3m	3m37s	+ (0.00)	Not found	0	2.75	1
gmail e. (POP3)	7/20	8/20	50m51s	38m11s	~ (0.24)	Not found	0	4.35	1
gmail f. (POP3)	20/20	19/20	3m	3m10s	+ (0.00)	Not found	0	3.25	1
AVERAGE	15.7/20	13.2/20	19m22s	24m18s		N/A	0	2.9	187

TABLE I: **Speed and precision results** of our TULIP tool, as well as ROSA and STRINGER, to detect the 14 backdoors from our benchmark. A discussion of these results can be found in Section II-B on page 3.

= 1.0), i.e., no false alarms were produced. By comparison, ROSA attains a lower recall (0.66, 185/280), but at the cost of substantially more false alarms (precision = 0.19, average of 2.9 false alarms per [run / detected backdoor]). Overall, these results emphasize that TULIP provides more actionable alerts, which is crucial in expert-driven backdoor detection settings.

Overall, the results suggest that the protocol-aware guidance of TULIP tend to make backdoor detection usually faster, compared to relying on a general-purpose fuzzing approach as in ROSA. This is overwhelmingly true for unchecked authentication backdoors, where the backdoor is detected by TULIP using the simple adversarial commands, so that no fuzzing is needed and detection is immediate. Yet, when the backdoor trigger diverges significantly from the normal syntax of the protocol, the protocol-aware guidance of TULIP becomes less beneficial (e.g. gmail e.) or even detrimental (e.g. PHP). Indeed, for the PHP backdoor, the fuzzer must come up with a syntactically invalid variant of an HTTP request containing some valid PHP code to trigger the backdoor. In this case, the adversarial seeds and restricted mutators of TULIP divert a lot of the fuzzing energy on a syntactically-valid but backdoor-free part of the pure HTTP input space, penalizing so TULIP against ROSA.

Success and failure patterns. TULIP is extremely efficient for unchecked authentication backdoors (ProFTPD 1.3.5, as well as gmail a. and b.), where the backdoor is detected by TULIP using the simple adversarial sequences, so that no fuzzing is needed and detection is immediate. Yet, when the backdoor trigger diverges significantly from the normal syntax of the protocol, the protocol-aware guidance of TULIP may become less beneficial (e.g. gmail e.) or even detrimental (e.g. PHP). Indeed, for the PHP backdoor, the fuzzer must come up with a syntactically invalid variant of an HTTP request containing some valid PHP code to trigger the backdoor. In this case, the adversarial seeds and restricted mutators of TULIP divert a lot of the fuzzing energy on a syntactically-

valid but backdoor-free part of the pure HTTP input space, penalizing so TULIP against ROSA.

C. Threats to validity

One category of threats to the internal validity of our findings stems from potential flaws in the software tools and manual procedures we utilized. Nevertheless, AFL++ is a widely adopted, open-source fuzzer maintained by the community, and frequently serves as a benchmark in fuzzing research. Our TULIP tooling and the broader experimental setup are open-source, they have undergone unit testing and have been validated through smaller-scale fuzzing trials. Another source of internal validity concerns lies in the inherent randomness of fuzzing, which could render our results statistically insignificant. To address this, we conducted 20 independent 1-hour runs, averaged the outcomes and reported p-values, thereby reducing the impact of stochastic variability.

As with all empirical research, this study faces the challenge of limited generalizability. To mitigate this, we executed our experiments across 14 backdoors—comprising 7 real-world and 7 synthetic examples—embedded within standard network protocol implementations. While it remains possible that a sophisticated backdoor trigger might evade detection by TULIP, the tool nonetheless raises the difficulty for attackers, likely compelling them to design more complex and thus more difficult-to-inject backdoors.

D. Conclusion

TULIP achieves a high backdoor detection rate while maintaining a very low false-alarm rate. These results highlight the benefits of our protocol-aware approach: in our experiments, ROSA and STRINGER exhibit lower detection effectiveness and substantially higher false-alarm rates. This difference is critical in practice, since false alarms impose significant manual triage and reverse-engineering effort.

REFERENCES

- [1] Earl T. Barr, Mark Harman, Phil McMinn, Muzammil Shahbaz, and Shin Yoo. The Oracle Problem in Software Testing: A Survey. *IEEE Transactions on Software Engineering*, 41(5):507–525, May 2015.
- [2] Eloi Benoist-Vanderbeken. Some codes and notes about the backdoor listening on tcp-32764 in linksys wag200g, 2015. <https://github.com/elvanderb/TCP-32764/tree/master> [Accessed: May, 22, 2024].
- [3] /dev/ttyS0 Craig. From china, with love, 2013. <https://web.archive.org/web/20131020145741/http://www.devttys0.com/2013/10/from-china-with-love> [Accessed: May, 22, 2024].
- [4] Chris Evans. Alert: vsftpd download backdoored, 2011. <https://scarybeastsecurity.blogspot.com/2011/07/alert-vsftpd-download-backdoored.html> [Accessed: July, 19, 2024].
- [5] Andrea Fioraldi, Dominik Maier, Heiko Eißfeldt, and Mark Heuse. AFL++: Combining Incremental Steps of Fuzzing Research. In *WOOT’20: Proceedings of the 14th USENIX Conference on Offensive Technologies*, page 10. August 2020.
- [6] Tom Ganz, Inaam Ashraf, Martin Härterich, and Konrad Rieck. Detecting Backdoors in Collaboration Graphs of Software Repositories. In *Proceedings of the Thirteenth ACM Conference on Data and Application Security and Privacy*, pages 189–200, Charlotte NC USA, April 2023. ACM.
- [7] Patrice Godefroid. Fuzzing: Hack, art, and science. *Communications of the ACM*, 63(2):70–76, January 2020.
- [8] Mark Harman and Phil McMinn. A theoretical and empirical study of search-based testing: Local, global, and hybrid search. *IEEE Transactions on Software Engineering*, 36(2):226–247, 2009.
- [9] George Klees, Andrew Ruef, Benji Cooper, Shiyi Wei, and Michael Hicks. Evaluating fuzz testing. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security, CCS ’18*, page 2123–2138, New York, NY, USA, 2018. Association for Computing Machinery.
- [10] Dimitri Kokkonis, Michael Marcozzi, Emilien Decoux, and Stefano Zacchiroli. ROSA: Finding Backdoors with Fuzzing . In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*, pages 2816–2828, Los Alamitos, CA, USA, May 2025. IEEE Computer Society.
- [11] Dimitri Kokkonis, Michaël Marcozzi, Emilien Decoux, and Stefano Zacchiroli. The ROSARUM backdoor detection benchmark, 2025. archived on Software Heritage with SWHID swh:1:rev:21d986293f083a09c0692c504305ac6e4fb9bf38.
- [12] ZDNET.com Michael Lee. D-link routers found to contain backdoor, 2013. <https://www.zdnet.com/article/d-link-routers-found-to-contain-backdoor> [Accessed: May, 22, 2024].
- [13] M. Schloegel, N. Bars, N. Schiller, L. Bernhard, T. Scharnowski, A. Crump, A. Ale-Ebrahim, N. Bissantz, M. Muench, and T. Holz. Sok: Prudent evaluation practices for fuzzing. In *2024 IEEE Symposium on Security and Privacy (SP)*, pages 140–140, Los Alamitos, CA, USA, may 2024. IEEE Computer Society.
- [14] Felix Schuster and Thorsten Holz. Towards reducing the attack surface of software backdoors. In *Proceedings of the 2013 ACM SIGSAC Conference on Computer & Communications Security - CCS ’13*, pages 851–862, Berlin, Germany, 2013. ACM Press.
- [15] Yan Shoshitaishvili, Ruoyu Wang, Christophe Hauser, Christopher Kruegel, and Giovanni Vigna. Fomalice - Automatic Detection of Authentication Bypass Vulnerabilities in Binary Firmware. In *Proceedings 2015 Network and Distributed System Security Symposium*, San Diego, CA, 2015. Internet Society.
- [16] Sam L Thomas, Tom Chothia, and Flavio D Garcia. Stringer: Measuring the importance of static data comparisons to detect backdoors and undocumented functionality. In *Computer Security–ESORICS 2017: 22nd European Symposium on Research in Computer Security, Oslo, Norway, September 11-15, 2017, Proceedings, Part II 22*, pages 513–531. Springer, 2017.
- [17] Sam L. Thomas and Aurélien Francillon. Backdoors: Definition, Deniability and Detection. In Michael Bailey, Thorsten Holz, Manolis Stamatogiannakis, and Sotiris Ioannidis, editors, *Research in Attacks, Intrusions, and Defenses*, volume 11050, pages 92–113. Springer International Publishing, Cham, 2018.
- [18] Sam L. Thomas, Flavio D. Garcia, and Tom Chothia. HumIDIFY: A Tool for Hidden Functionality Detection in Firmware. In Michalis Polychronakis and Michael Meier, editors, *Detection of Intrusions and Malware, and Vulnerability Assessment*, volume 10327, pages 279–300. Springer International Publishing, Cham, 2017.
- [19] John Toterhi. Hunting for backdoors in iot firmware at unprecedented scale. In *Proceedings of the 2018 Hack in the Box Dubai Hacking conference Security - HITBSecConf Dubai ’18*, 2018.
- [20] Xiaohan Zhang, Cen Zhang, Xinghua Li, Zhengjie Du, Bing Mao, Yuekang Li, Yaowen Zheng, Yeting Li, Li Pan, Yang Liu, et al. A survey of protocol fuzzing. *ACM Computing Surveys*, 57(2):1–36, 2024.